

redis-hands-on

Redis - Hands-On Concepts

Redis is an in-memory key-value store. It is extremely fast because all data lives in RAM. It is primarily used for caching, session storage, rate limiting, and pub/sub messaging - not as a primary database.

1. Core Concepts to Understand First

Term	What it means
Key	A unique string that identifies a value
Value	The data stored under a key (can be a string, list, set, hash, etc.)
TTL	Time To Live - how long a key exists before Redis deletes it automatically
In-memory	All data is stored in RAM, making reads and writes extremely fast
Persistence	Redis can optionally save data to disk, but it is not its primary purpose

2. Connecting via CLI

```
redis-cli
```

Test the connection:

```
PING
# Response: PONG
```

3. Strings - The Most Common Data Type

Strings are the simplest and most used type. A value can be plain text, a number, or a serialized JSON string.

```
# Set a key
SET username "alice"

# Get a key
GET username

# In Redis, commonly use colon : separated keys as a naming convention (like namespaces)
# Set with expiry (TTL in seconds)
SET session:cookies "user_data" EX 3600

# Check how many seconds remain before a key expires
TTL session:cookies

# Delete a key
DEL username

# Check if a key exists (returns 1 if yes, 0 if no)
EXISTS username
```

4. TTL - Expiry Management

TTL is one of the most important features in Redis. It is how caching and session expiry work.

```
# Reinsert username
SET username "alice"

# Set expiry on an existing key (in seconds)
EXPIRE username 600

# Set expiry in milliseconds
PEXPIRE username 60000

# Remove the expiry (make the key permanent)
PERSIST username

# Get remaining TTL
TTL username      # Returns seconds (-1 means no expiry, -2 means key does not exist)
```

5. Lists

Lists are ordered sequences of strings. Useful for queues and activity feeds.

```
# Push to the left (start) of a list
LPUSH notifications "message1"

# Push to the right (end) of a list
RPUSH notifications "message2"
RPUSH notifications "message3"

# Get a range of elements (0 = first, -1 = last)
LRANGE notifications 0 -1

# Get the length of a list
LLEN notifications

# Pop from the left (remove and return)
LPOP notifications

# Pop from the right
RPOP notifications
```

6. Hashes

Hashes store an object as a set of field-value pairs under a single key. Think of it like storing a row of data.

```
# Set fields on a hash
HSET user name "Alice" age "25" role "intern"

# Get a single field
HGET user name

# Get all fields and values
HGETALL user

# Check if a field exists
```

```
HEXISTS user email

# Delete a field
HDEL user age
```

This is useful when you want to cache a user profile or any object without serializing it to JSON.

7. Sets

Sets are unordered collections of unique strings. Useful for tracking unique items - like users who have seen a notification.

```
# Add members to a set
SADD active_users "alice" "bob" "carol"

# Get all members
SMEMBERS active_users

# Check if a value is in a set
SISMEMBER active_users "alice"

# Remove a member
SREM active_users "bob"

# Count members
SCARD active_users
```

8. Sorted Sets

Sorted sets are like sets but every member has a score. Members are automatically ordered by score. Useful for leaderboards and priority queues.

```
# Add members with scores
ZADD leaderboard 1500 "alice"
ZADD leaderboard 2300 "bob"
ZADD leaderboard 800 "carol"

# Get members ordered by score (ascending)
ZRANGE leaderboard 0 -1 WITHSCORES

# Get members ordered by score (descending)
ZREVRANGE leaderboard 0 -1 WITHSCORES

# Get the rank of a member (0-indexed)
ZRANK leaderboard "alice"

# Get the score of a member
ZSCORE leaderboard "bob"
```

9. Pub/Sub - Publish and Subscribe

Pub/Sub allows one part of your system to publish a message and another part to receive it in real time. Useful for notifications and event-driven workflows.

Open two separate terminal windows connected to `redis-cli`.

Terminal 1 - Subscribe to a channel:

```
SUBSCRIBE alerts
```

Terminal 2 - Publish a message:

```
PUBLISH alerts "Server restarted"
```

Terminal 1 will instantly receive the message.

10. Useful Utility Commands

```
# List all keys (avoid in production on large datasets)
KEYS *

# Search keys by pattern
KEYS session:*

# Get the data type of a key
TYPE username

# Get total number of keys in the current database
DBSIZE

# Delete all keys
FLUSHDB
```

11. Things Worth Knowing

- Redis is not a replacement for a primary database. Use it alongside MongoDB, not instead of it.
- Always set a TTL on cached data. Keys without expiry accumulate and can exhaust memory.
- Data in Redis is lost if the server restarts and persistence is not configured. Design accordingly.
- Redis is single-threaded for command execution, which is why operation order is predictable and atomic.